es

Threat modeling

Refer to the threat modeling runbook page.

Time estimation

Based on the defined scope and the "size" of the feature as well as any time constraints listed in the requesting issue the reviewer should be able to give a estimation for the review to be completed. Under tight time constraints a second (or more) reviewer might be requested to conclude the review in a timely manner.

Preparation Output

Output of the preparation step should be captured in the review issue using the Preparation section in the issue template(internal link). The table of to-be-tested in-scope items should be kept up to date whenever a test item has been addressed. The coverage percentage should be estimated and noted in the according column as well. This table might be a high-level overview only but should contain all relevant topics. Further refinement of the to-be-tested items will be done in the subsequent steps. The time estimation should be filled out in the template text and the review issue due date should also be set to the expected end date of the review.

Review process

Following the in-scope items by priority the reviewer will conduct the requested review. Any security related findings and concerns should be captured in a dedicated thread in the review issue. Having all initial security concerns and findings within the issue allows the review issue to be a single source of truth for its review cycle. The individual findings can be discussed in the separate threads with the development team, if the issue needs a dedicated in-depth discussion a separate issue should be created in the according project and linked to the review issues. In order to close a
review issue all findings need to be addressed and follow up issues need to be created on open discussions.

Finding description

Each finding or security concern should get its own thread in the review issue. The following template can be used:

markdown
 short description
 Severity: Informational/Low/Medium/High/Critical

details and reproduction steps

The heading should contain a concise description of the issue. For instance "Reflected Cross-Site Scripting in search parameter" or "Lack of certificate validation in backend communication". The details should go in-depth such that the issue is clearly understandable and, if applicable, can be reproduced with reasonable effort.

Not directly exploitable issues like missing or incomplete standards, such as the lack of usage of Secure tools on all production code are also to be documented as a finding during the review process.

Conclusion

When the review is done a conclusion section should be added as a comment to the issue using the Conclusion section in the issue template.

The conclusion section should contain a brief summary of the findings, potentially highlighting any critical findings. When the review process itself went very well this section should also be used to point out the participating team members for their
collaboration
efforts.

Coverage

The coverage section should be used to note how well the scope was covered, what has been tested and what could not be reviewed. If possible also test and review steps which did not produce a finding should be noted as well.

Findings

The findings section should contain a list of all findings made during the review. The findingtable.rb script will try to
pre-populate a table for this. The Remediation column still needs to be filled out manually to point to the according remediation MRs or issues.

Problems

If any blockers or problems occurred during the review those should be mentioned as well. In order to stay
results oriented any
problem should come with a suggestion how to do it better or how to avoid the problem in future reviews.

Next review cycle

Usually the reviewed component is not finished in way that would allow to sign off a security check forever. Therefore the last section in the conclusion should mention a reasonable time frame for a follow up review. This might be shortly before a feature or component is launched or when substantial changes

are being made to the component.

AppSec follow up actions

Follow up activities for the AppSec team such as "improve best practices documentation for X", where a common usage pattern is identified should be listed here. A respective issue should be filed for any AppSec follow up task and linked in this section.

Closing the review issue

It might be the case that over the timeline of the review some findings already are getting addressed, some might not yet be considered by the development team.

In order to close the review issue there should be either a follow up issue or MR on the respective development repository. If a follow up is missing it should be created by the reviewer and linked to in the respective finding thread. For those issues the normal triage process
applies.

SECTION: security/product-security/application-security/runbooks/scw-engagement-plan.md

How can AppSec Engineers Contribute to the Secure Code Warrior Training Program?

If anyone from the AppSec team is interested in contributing to the Secure Code Warrior (SCW) training program and you don't have access to the Secure Code Warrior training portal, please post a comment in the sec-appsec Slack channel requesting access.

Once you have access to the Secure Code Warrior training portal, please do the following:

- Join the security-secure-code-warrior Slack channel
- Watch Secure Code Warrior platform walkthrough for a quick overview of the platform
- Please read through SCW's documentation on Getting Started
- Go through our old SCW Trial issue to get an idea on how we have organized Tournaments in SCW.

Once you have done the above, you will be in a great shape to start contributing to the SCW training program. Please feel free to post in sec-appsec if you have any questions.

Content Development

Any GitLab team member is welcome to contribute to the development of course content. AppSec should ensure that content development involves team members who will take the course, particularly if that course is mandatory. For example, involve Frontend Engineers in the development of a course that includes Vue.JS content.

When developing content, consider:

- What is the learning focus? E.g. broad framework security awareness vs. specific training on a specific class of vulnerability
- How long should the course take? This impacts the content included: the number of videos, challenges, etc
- Is the content high quality? Work with team members and our Secure Code Warrior customer support manager to ensure the training provides good advice and bug-free challenges

Engagement Goal

Developer = Backend + Frontend Engineers. See Org Chart for more information on the number and types of developers at GitLab.

- 90% of developers being onboarded create an account in Secure Code Warrior
- 80% of developers complete relevant training within 3 months of starting
- 25% of developers participate in the semi-annual tournament
- 50% of developer accounts are spending at least an hour in the platform each month

Later

- 75% of developers in high risk teams ("high risk" is defined as teams or groups that see the most amount of vulnerabilities reported in their part of the application in a given year) have completed one course, per year, in SCW

Engagement Plan

- Integrating Secure Code Warrior into new hire onboarding template as a checklist item that needs to be finished within the first 3 months of starting at GitLab
  - An Access Request (AR) would be created automatically requesting the new hire access to Secure Code Warrior and the request would ping the @gitlab-com/gl-security/product-security/appsec DL on the AR issue. The AppSec person on triage rotation picks up the AR.
- Monthly Slack announcements calling out the top 3 vulnerabilities seen in our last Security Release, announcement to be made by the AppSec Primary for that release (see Release Managers), and, how developers can learn to identify and fix them
- Integrating Secure Code Warrior with GitLab CI would greatly help drive continuous engagement
- Kick-off tournament open to all Development engineers at GitLab. Prizes would be given for the top 3 players of the tournament.

How to measure what, when, and, why?

[Trailing, 3 months] Integrating Secure Code Warrior into new hire onboarding template as a checklist item that needs to be finished within the first 3 months of starting at GitLab

- Measure the number of new hires that sign up on Secure Code Warrior
  - Measure monthly
- Measure the number of new hires that finish the SCW onboarding course within 3 months
  - Measure monthly
- Measure how many left the course midway
  - Measure monthly
- For the users who signed up, measure how many hours they spent in the platform
  - Measure monthly

- For the users who signed up, measure how many activities they did in the platform
  - Measure monthly

[Trailing, 1 month] Those who sign in to Secure Code Warrior are able to use it successfully

- Across all users, measure how many hours they spent in the platform
  - Measure daily, review monthly
- Across all users, measure how many activities they did in the platform
  - Measure daily, review monthly

[Leading & Trailing] Monthly Slack announcements calling out top 3 vulns seen in our last Security Release and how developers can learn to identify and fix them

- How many people are reacting to the Slack announcement?
  - Measure when we make the Slack announcement
- Are the number of new people signing up on the platform increasing after the Slack announcement?
  - Measure when we make the Slack announcement
- Is the existing user engagement increasing after the Slack announcement?
  - Measure when we make the Slack announcement
- Are developers doing the same challenges from vulnerability areas identified in the Slack message? If not, does the Slack announcement need to be tweaked to encourage more engagement?
  - Measure when we make the Slack announcement

[Trailing, 1 month] Integrating Secure Code Warrior with GitLab CI would greatly help drive continuous engagement

- How many developers are clicking on the training link on the security issue created by SCW?
  - Measure monthly
- Those clicking on the link, are they completing the training on SCW?
  - Measure monthly
- What feedback are we getting from developers on this integration?
  - Measure monthly
- How should we capture and improve on this feedback?
  - Measure monthly

[Trailing, 6 months] Semi-annual tournaments for all developers at GitLab

- How many developers are signing up for the semi-annual tournament?
  - Measure semi-annually (during the tournament)
- How engaged are the developers on the platform?
  - Measure semi-annually (during the tournament)
- What feedback are we getting from developers on the tournament?
  - Measure semi-annually (during the tournament)
- How should we capture and improve on this feedback?
  - Measure semi-annually (during the tournament)

SECTION: security/product-security/application-security/runbooks/security-dashboard-

review.md

Frequency: Daily

AppSec engineers are responsible for triaging the findings of the GitLab security tools. This role has two primary functions.

1. Validate the findings and handoff to engineering for correction
1. Provide feedback to the Secure team

Security Dashboard List

The following is a list of security dashboards that need to be reviewed:

- GitLab
- GitLab Omnibus
- Gitaly
- GitLab Pages
- GitLab Shell
- k8s-workloads
- Version
- UBI images
- release-cli
- vscode-extension
- Customers
- elastic-search-indexer
- GitLab DAST
- GitLab Terminal
- GitLab Agent
- Compensation Calculator

Validation

For each finding:

- Using the information provided and any additional information, determine if the finding is valid
- For a valid report:
  - Change the status to Confirmed
  - Click Create Issue
  - Assign Priority and Severity labels based on the finding rating and the impact on GitLab
  - Assign a Due Date
  - add labels (/label ~ command) corresponding to the DevOps stage and source group (consult the Hierarchy for an overview on categories forming the hierarchy)
  - @-mention product manager of appropriate teams for scheduling and/or the engineering managers if additional engineering feedback is required to complete the triage, based on the product categories page
    - If an appropriate engineering team is not immediately apparent, ping an Appsec manager for help identifying the owner
- For an invalid report:

- Change the status to Dismissed
- Leave a comment providing feedback on why the vulnerability is being dismissed. This information can be used by Secure engineers in tuning the findings of the tool and is also useful if we ever wonder why it was dismissed at some point in the future.

Dependency Updates

If a vulnerability is identified in a product dependency, the appsec engineer should follow the security development workflow to create a merge request to update the dependency in all supported versions. The merge request should be opened in the GitLab Security repo so that the dependency gets updated in supported backports as well. Vulnerabilities determined to be Critical or High should have merge requests created when identified. Medium and Low vulnerabilities will be addressed by best effort, but always within the 90-day SLA.

The goal of this process is to update dependencies as quickly as possible, and reduce the impact on development teams for minor updates. In the future, this step could be replaced by auto remediation.

If an upgrade to a new major version is required, it might be necessary for the update to be handled directly by the responsible development team.

- Complete the Validation process above.
- Create an issue in the GitLab Security repo issue tracker using the Security developer workflow template.
- Follow the template to create a merge request targeting the master branch.
- If the resulting pipeline is clean, continue with the developer checklist in preparation for merging. This must include reviews by a reviewer and maintainer.
- If the pipeline fails, work with the responsible engineering team to remediate. Depending on the cause, it may be necessary for a developer with advanced knowledge of the dependency functionality to review the change. If a developer is not immediately available, continue to track the issue/MR to ensure remediation within the defined SLA.

SECTION: security/product-security/application-security/runbooks/security-engineer.md

{{% include-remote "https://gitlab.com/gitlab-org/release/docs/-/raw/master/general/security/security-engineer.md" 1 %}}

SECTION: security/product-security/application-security/runbooks/threat-modeling.md

:warning: Prioritization Note

As of 2023-11-02, AppSec is only prioritizing P1 AppSec reviews and threat models. This does not mean P2 or P3 AppSec reviews or threat model requests can't be submitted, but please understand that due to capacity limitations we will only be able to prioritize P1 reviews. Reach out to us in the sec-appsec Slack channel if you have any questions or concerns.

AppSec wants to encourage teams to create their own threat models, and the team will try to support any queries in regards to the process.

Using the threat model scoped labels

To assist in the creation of threat models the two labels ~threat model::needed and ~threat model::done
should be used. Whenever a threat model for a particular issue or epic should be created the Appsec stable
counterpart will apply the ~threat model::needed label to the epic or issue. The Appsec stable counterpart
will also create a dedicated threat modeling issue in the
threat-models(internal link) project.
Within the issue the development and the application security team should collaborate on the creation
of a threat model. The development team must provide technical documentation in the issue before the process starts. The issue template contains detailed steps to guide through the process.

If you're new to threat modeling: for a beginner friendly start please have a look at our threat modeling how to page.

Creating the threat model

The scope definition and prioritization is a very first step towards
building a proper threat model
for the to-be-reviewed item. If time allows and the complexity of
to-be-reviewed feature justifies it, a more in-depth threat model
should be developed. In the context of an AppSec review we will
follow the PASTA approach
which has been chosen to be the most flexible approach fitting
for various threat modeling cases throughout GitLab.

- In the AppSec review we should start with the Stage III - Application Decomposition
  as soon as we have a clear scope definition for the review. This stage
  of application decomposition is an extension of the prioritization step
  where much more details should be considered.
- The threat analysis Stage IV - Threat Analysis
  should be used to create a detailed test plan for the review.
- Finally the
  Stage V - Vulnerability and Weakness Analysis
  would be the actual technical review process which allows us to verify
  the assumptions of the two previous stages.

In total this would reflect the
evidence driven threat model
within the review.

Stage III - Application Decomposition

This stage of the threat modeling approach is listed in the review template as a dedicated item. In this stage, the most work is being done to actually have a model of what is being reviewed.

For the typical AppSec review of a new GitLab feature a few things should be considered to get a meaningful threat modeling output. The threat model should be developed starting at the entry points of external data considering any security or trust boundaries the data passes. This will result in a of Data Flow Diagram which can be used to guide the actual review process. In the threat modeling context a Data Flow Diagram is a visual representations of the modeled item, showing the components and their interaction along with trust boundaries which might be traversed by data form untrusted sources.

A good starting point for a data flow diagram in threat model is usually to take (a maybe existing) architecture diagram, like to following from the GitLab Agent for Kubernetes:

```
mermaid
graph TB
  agentk -- gRPC bidirectional streaming --> nginx

  subgraph "GitLab"
  nginx -- proxy pass port 5005 --> kas
  kas[kas]
  GitLabRoR[GitLab RoR] -- gRPC --> kas
  kas -- gRPC --> Gitaly[Gitaly]
  kas -- REST API --> GitLabRoR
  end

  subgraph "Kubernetes cluster"
  agentk[agentk]
  end
```

Considering this diagram we can already spot a trust boundary between the agentk and the GitLab components. The flow of data is also depicted in a usable way for threat modeling and the involved components which consume or provide data are visible. Such a diagram can now be used and complemented with actual threat and security context to form an actual threat model.

More specific instructions and steps are listed in the review template.

Stage IV - Threat Analysis

This stage is the actual doing of the review. Any findings made are the main output of the threat analysis.

Stage V - Vulnerability and weakness analysis

The output of the review process done in stage IV should be mapped back to the

decomposition in stage III. That way we can verify that our decomposition and the initial threat assumptions did provide meaningful guidance for the review process. The output and mapping can be documented in the conclusion's summary and coverage sections.

SECTION: security/product-security/application-security/runbooks/triage-rotation.md

Application Security team members are alphabetically assigned as the responsible individual (DRI) for incoming requests to the Application Security team, typically for a weekly or fortnightly period.

Who is on rotation?

Automation manages the scheduling and assignment of rotations:

- "Who is on rotation now?"
- Rotation Management FAQ & README
- Holiday Coverage

What are the rotations?

The following rotations are defined:

- (Weekly Assignment) HackerOne + Security Dashboard Review
  - Point of contact for "New" HackerOne reports during that week.
  - Responsible to escalating to other team members and management if the size of the either queue spikes.
  - Responsible for reviewing security dashboards on a best-effort level
- (Weekly Assignment) Triage Rotation (mentions and issues), by order of priority:
  - First responder to JiHu Contribution pings that come into the sec-appsec Slack channel
  - First responder to automated messages posted in the publicmergerequestsreferencingconfidentialissues Slack channel
    - Add a check mark emoji if the merge request can be public
    - If the merge request references a legitimate security issue
      - If the issue has a ~security-fix-in-public label, indicating it has been approved by an AppSec team member to be fixed in public, link to the comment granting approval or include a message in Slack denoting that the ~security-fix-in-public label was added.
      - Decide if it can be public anyway, and apply the ~security-fix-in-public label retrospectively
      - Otherwise contact SIRT and the merge request author to get the merge request removed.
      - Use the Urgent - SEOC should be paged right away option if waiting up to 24 hours for a resolution would be too long.
  - First responder to mentions of the following group aliases:
    - @gitlab-com/gl-security/product-security/appsec on GitLab.com
      - PSIRT and/or SIRT are responsible for addressing external reports of a product vulnerability or customer exploit. See Hand-off to PSIRT/SIRT during triage rotation
    - @appsec-team in Slack
  - First responder to mentions from the custom SAST bot:

    - All merge requests with the ~appsec-sast-ping::unresolved label must be reviewed.
    - Apply the ~appsec-sast-ping::resolved label once the bot's findings have been resolved.
    - A dashboard with all the bot's findings can be found here.
  - First responder for issues created needing triage: ~security-triage-appsec issue search
    - Refer to this page to learn about the different labels that we can apply to issues when they're not vulnerabilities
- (~Fortnightly Assignment) Security Engineer for Security & Patch Releases
- (Fortnightly Assignment, Federal AppSec only) Release Certifications
  - Responsible for the release certification process
  - This applies to any release that might have JiHu contributions, including monthly and patch releases
- (Quarterly Assignment) Bug Bounty/AppSec Blog Post

If the Application Security team member has a conflict for the assigned week they may swap rotation weeks with another team member. This may be done for any reason including time off or the need for time to focus on a particular task.

Team members should not be assigned on weeks they are responsible for the scheduled security release.

Team members not assigned as the DRI for the week should continue to triage reports when possible, especially to close duplicates or handle related reports to those they have already triaged.

Team members remain responsible for their own assigned reports.

Hand-off to PSIRT/SIRT during triage rotation

When team members are assigned to Triage rotation and are first responder to mentions of @gitlab-com/gl-security/product-security/appsec on GitLab.com or @appsec-team in Slack, assess whether the ping is an external report of a product vulnerability or customer exploit. In these instances, hand off to @gitlab-com/gl-security/product-security/appsec/psirt-group and/or @gitlab-sirt.

Direct reports from customers of vulnerabilities found during container scans to the Vulnerability Mangement team.

Triaging exposed secrets

Exposure of information and secrets is handled a little differently to vulnerabilities, as there is nothing to patch and therefore no need for a GitLab Project Issue, CVSS, or CVE. When you're pinged during your rotation and you see a leaked secret, follow the process described on the HackerOne runbook

---
SECTION: security/product-security/application-security/runbooks/unintended-vuln-disclosure.md

Summary

There are situations where details about an unfixed (or insufficiently fixed) vulnerability can be inadvertently made public. This runbook includes specific processes for AppSec to follow in these situations.

Instructions for paging SIRT

For actions outlined below that involve paging SIRT, please check the "page immediately box" for issues with severity above ~severity::4.

Vulnerabilities fixed in canonical instead of security mirrors

Immediate response

1. Check whether the security issue is OK to be addressed in public. This is usually denoted by ~"security-fix-in-public" label and if this label is not there proceed to next step.
1. The AppSec engineer should page SIRT by using /security in Slack.

Mitigation

1. SIRT performs a commit cleanup

Follow-up actions

1. SIRT performs monitoring to look for active exploitation attempts against the vulnerability.

Insufficient fix of a different or related vulnerability

Immediate response

1. If the vulnerability is critical, SIRT must be paged via /security so they can begin working on detection and mitigation.
1. AppSec should create a new issue with the appropriate priority and severity labeling. The appropriate EM / PM should be pinged so they can work on it immediately.

Mitigation

1. Same as any other vulnerability report.

Follow-up actions

1. If the fix was found out to be insufficient shortly after it went out, consider opening an AppSec review for that feature.
1. Consider opening an RCA issue to figure out why the fix was insufficient.

HackerOne reporter decides to make something public prior to a fix being made

Immediate response

1. The AppSec engineer should immediately page SIRT by using /security in Slack.

Mitigation

1. Bump up the SLA windows to urge the developers to work on fixing the vulnerability on priority.
1. Consider scheduling a security release, regular or critical - depending on the severity of the vulnerability.

Follow-up actions

1. Ban the HackerOne researcher from the program following the code of conduct violation process.
1. Communicate the situation to the Legal team via the legal Slack channel or pinging them into the issue, so they can determine what (if any) steps need to be taken from a legal perspective
1. Involve Communications to alert any potentially impacted customers.

Communication mistakes or other unintentional leaks

For the following:

- Titles revealed via email inbox leak in a YouTube video
- Someone accidentally gives too much information to a community member or customer

Immediate response

1. Figure out if enough information was leaked for an attacker to build a PoC to exploit the vulnerability. If yes, page SIRT by using /security in Slack.

Mitigation

1. Make the YouTube video private by following these instructions.
1. Communicate with the EMs, PMs, and development teams to urge them to work on fixing the vulnerability as a priority.
1. Consider starting a conversation with Delivery, Dedicated, and the relevant development teams about potentially scheduling an ad-hoc critical security release, depending on the severity of the vulnerability.

Follow-up actions

1. Reach out to the team member that caused the leak, and educate them on keeping vulnerability-related data SAFE.

For the following:

- Vulnerability issue wasn't made as confidential, or accidentally becomes visible to everyone
- Accidentally added to the security release blog post even though the fix didn't make it into the release

Immediate response for Communication mistakes or other unintentional leaks

1. Change the confidentiality of the issue.

1. Edit the blogpost and ping the SRE on-call to merge it right away.
1. Page SIRT by using /security in Slack.

Mitigation for Communication mistakes or other unintentional leaks

1. Communicate with the EMs, PMs, and development teams to urge them to work on fixing the vulnerability as a priority.
1. Consider starting a conversation with Delivery, Dedicated, and the relevant development teams about potentially scheduling an ad-hoc critical security release, depending on the severity of the vulnerability.

Follow-up actions for Communication mistakes or other unintentional leaks

1. Was the confidentiality of the issue incorrectly changed by a team member? If yes, educate them on keeping vulnerability-related data SAFE.
1. Ping the AppSec release managers about the incident, as before the release goes out, it is carefully vetted for such mishaps.
1. Addition of vulnerability details to the blogpost is automated by the Delivery team. Reach out to them to figure out if this was somehow a bug in their tooling.

SECTION: security/product-security/application-security/runbooks/upstream-security-patches.md

{{% include-remote "https://gitlab.com/gitlab-org/release/docs/-/raw/master/runbooks/security/upstreamsecuritypatches.md" 1 %}}

SECTION: security/product-security/application-security/runbooks/verifying-security-fixes.md

The review of a fix by an application security engineer is triggered by the engineer implementing the fix. The security engineer that triaged the vulnerability report is responsible for validating the merge request involving the fix. As the engineer engaging in the verification, these are the steps that should be followed:

1. Doing a code review from the security perspective.
1. Validating the security fix using the Docker image generated by package-and-qa, see the section below for more details.
1. Once the validation is done, update the security issue with a link to the CVE.

   - The link goes in the Summary > Links table of the security implementation issue, next to the cell labeled CVE ID request
   - If the fix does not require a CVE, post in the security implementation issue that no CVE will be requested with the reason why.
   - If a CVE was created on import from HackerOne, double check the submission contains up-to-date details

Note: Approval is only required for the merge request targeting master, it's not required for

the merge requests
targeting a versioned stable branch (X-Y-stable-ee) .
Note: The process described above is particular to validate GitLab fixes. The process to verify fixes for other subcomponents (Omnibus GitLab, Gitaly, and GitLab Pages) might be different.

Validating security fixes using Gitpod

1. Start a new Gitpod instance https://gitlab.com/gitlab-org/gitlab-development-kit/-/blob/main/doc/howto/gitpod.md
1. When your Gitpod instance has started, download the patch from the security MR by clicking on the Code option on the top right of the MR.
1. In your Gitpod instance on the terminal: cat > mr.patch <hit return> <paste contents> <hit control+d>. Then apply the patch: git apply mr.patch
1. Restart GDK on Gitpod: gdk restart
1. Make the GitPod webserver reachable: In the Ports pane of Gitpod, find the row with port 3000 and click the lock icon to make the port public.
In the same Ports pane, and the same port 3000 row, click the URL to visit the GitLab instance.

Validating security fixes using package-and-qa

Note: The Delivery team recorded a video about summarizing this process, it's highly encouraged for AppSec team members
to see it.

The package-and-qa build runs end-to-end tests against an Omnibus package that is generated from the merge request changes.
In order to validate that the security fix introduced on the merge request remediates the vulnerability, Security engineers will use this package
to spin up a docker image in their local environment.

For that, Security Engineers need to follow these steps:

1. On the security merge request, click on the qa stage and then trigger the e2e:test-on-omnibus build.
   - If the QA Stage docker build is not present, label the MR with ~"pipeline:run-all-e2e" to initiate the pipeline in the QA stage that will generate the docker image. Be sure to kick off a new pipeline. For more info see this handbook page.
1. Internally the e2e:test-on-omnibus build will trigger a pipeline on the [Omnibus GitLab Mirror] project.
1. On the Omnibus GitLab Mirror pipeline, wait for the Trigger:gitlab-docker build to finish.
1. In the meantime,
   - Ensure you're logged in to registry.gitlab.com. You can login with your pre-configured Docker credentials,
     or with a Personal Access Token or a Deploy Token using the command docker login registry.gitlab.com (or nerdctl login registry.gitlab.com -u <username> depending on what you're using).
   - Complete the Set up volumes location on the Omnibus
1. Once Trigger:gitlab-docker has been completed, scroll down to the end of the log
and find the docker image that was pushed to registry.gitlab.com.
1. To start the docker image on your local environment, follow the documentation and replace

the gitlab/gitlab-ee:latest image with the one from the previous step.
1. Wait for the installation to be completed, and after that, you'll be able to access the local instance
by going to 0.0.0.0:80 in your browser.

Requesting CVEs

1. Visit the gitlab-org/cves repository issue tracker. You will be requesting CVEs by making issues using the templates in this repository.
1. For each vulnerability (CVSSv3 score > 0) affecting the self-managed version of GitLab, request a CVE identifier by creating a new issue and selecting the Internal GitLab Submission template. Please note that some merge requests included in the security release do not require a CVE identifier (e.g., only third-party dependency upgrades, gitlab.com-only issues, etc.).

- The CVSSv3 score should have been approved by at least two team members in the Bug Bounty Council issue. For reports that predate this process, do not hesitate to add a note to the current Council issue only to discuss the CVSSv3 score with other team members.

1. If a CVE identifier is almost certainly going to be needed, please check the 'Automatically request a CVE ID' box. This will make sure that a CVE identifier is requested at the beginning of the process.
1. For the Title of each issue, please use the respective vulnerability title that is going to be included in the blog post entry.
1. The reporter section refers to us as requesters of the CVE and not the vulnerability reporter, the default values for those fields should be used.
1. Fill in the remaining fields with the relevant information using the template or other previous submissions as examples. Please note that you will likely need to look up the appropriate cwe value (see CWE) and also use the CVSSv3 calculator to determine the impact.
1. Note: some information, such as fixedversions and solution, may not be readily available when the CVE request issue is created. Be sure to update the CVE issue once you have that information.
1. Submit the issue. It will be reviewed by the vulnerability research team. They will follow up with any questions and then submit it to be assigned a CVE identifier.
1. For each CVE request submission, link the relevant security issue to it. Once a CVE identifier is assigned, please also comment with that on the relevant issue and ensure that it gets included in the blog post.

CVE credit

Most vulnerabilities are credited either to a HackerOne researcher or a team member. These reports are always credited using the existing CVE template.

When a vulnerability is reported directly as a GitLab issue or by a customer via ZenDesk, provide an opportunity to be publicly credited:

text
Hello [contact],

We are preparing to patch the issue you reported. Would you like to be publicly credited with discovering the issue, and if so, how?

Some suggestions are @socialmediahandle, FirstName LastName, FirstName LastName from CustomerName, or the team at CustomerName. We can also format the credit with a link.

Our default credit will be This vulnerability was reported by a customer for customers and This vulnerability was disclosed using our Coordinated Disclosure Process. for other members of the wider GitLab community.

Updating CVEs

To update a CVE after it has been published, open a merge request in <https://gitlab.com/gitlab-org/cves> which modifies published/CVE-YYYY-IDIDID.json.
Ping gitlab-org/secure/vulnerability-research to review and merge. The Vulnerability Research team will then handle updating the CVE with MITRE and/or NVD.

SECTION: security/product-security/application-security/runbooks/working-with-sirt.md

This runbook is meant to help AppSec engineers who need to engage and work with SIRT to respond to a HackerOne report, discovered vulnerability, or other security incident.

Requirements for using /security to engage SIRT

If this is a P1S1, follow the P1S1 runbook and engage the security on-call.

When using /security to engage SIRT:

- Only use /security after we have validated the vulnerability or otherwise confirmed the incident
- Fill out the form as completely as you can
- Include a link to the issue or HackerOne report, a summary of what has occurred, and a description of what we are asking SIRT to help with
- Select the appropriate urgency (see below for some guidelines)

Situations where SIRT needs to be engaged

Note: This is not an exhaustive list, there may be other situations in which you need to engage SIRT.

- P1 vulnerabilities
- Publicly known P2 vulnerabilities with a publicly available exploit
- Secret leaks
- Vulnerabilities that could result in account takeover
- Vulnerabilities that lead to customer communications being required in addition the usual processes
- Security fixes made in public that should have been made against a security repository
- Personal data leaks
- DNS takeovers

Urgency guidelines

These are some guidelines for selecting the urgency in the /security form:

Type of incident	Urgency	Notes
Any kind of S1	Urgent (page right away)	Examples include critical vulnerabilities, exposure of red data, still-valid team member token leaks
Personal data leaks	Not Urgent (review within 24 hours)	Could be Urgent depending on the volume and the data
Public merge requests fixing vulnerabilities not labeled security-fix-in-public	See notes	S2 and above is likely Urgent, S3 and below is Not Urgent

For more information, consider viewing the SIRT incident classification and severity matrix page.

Getting SIRT attention without using /security

In some situations, we just want SIRT to be aware of something that is happening or may become an incident soon. An example of this would be a high severity unverified HackerOne report from a reliable HackerOne reporter.

Put a message in the sdsecurityoperations Slack channel with a brief description of the situation. You may consider using /sirtoncall to determine who is on-call and pinging them.

For HackerOne reports, you can also import the report into a GitLab issue and then mention @gitlab-com/gl-security/security-operations/sirt on it. Keep in mind this may not be appropriate, since it will ping all team members of SIRT.

Assisting with incidents in general

- Be ready to join the SIRT bridge or collaborate with SIRT asynchronously
- Assist in writing summaries, documenting reproduction steps, and keeping the timeline up to date
- Add information and context in the incident issue and discussion threads
- Update the incident issue with access timestamps, your IP address, and the IP address of any HackerOne reporters (and/or HackerOne triage team) to assist with log reviews

---
SECTION: security/product-security/application-security/threat-modeling/_index.md

Threat modeling is the process of taking established or new procedures, and then assessing it for potential risks. For most tech companies, this usually involves code and coding changes. However this process can be adapted to any situation where there is a potential risk, and is something that many of us do every day. Choosing the longer well-lit walk to your car as opposed to the short cut through the darkened alley. Looking both ways before crossing the street. This is something we often do by instinct.

Within the context of GitLab, there are different risks we evaluate. Will my code change

introduce a security vulnerability into the product? Will a radical new company policy being introduced create outside criticism and damage GitLab's reputation? When was the last time we evaluated this existing code library for vulnerabilities? What are the potential pitfalls our new sales campaign might face when moving into new markets? These are risks we deal with all of the time, just like every other company.

Getting Started

Here are a few resources to help get you started in threat modeling:

We've developed an issue template available here (private link) that you can use to create an issue documenting your threat model. It's required that Engineering provide technical documentation when creating a threat model issue. We also request that the application decomposition, use case, external entrypoints, trust levels, data flow diagram as well as the previous security issues (if any) sections are filled out.

The basis of our threat modeling is modeled after PASTA It should be noted that a full PASTA threat model is usually not required as it involves 7 steps, and in many cases only the steps 4, 5, and 6 are needed. To make it even easier, you can use STRIDE) to help define the threats.

Therefore we've included a beginner-friendly how-to guide to threat modeling which you should read if you're new to threat modeling. It includes a bit more detail about using STRIDE. If you need additional help, please ping the AppSec team or reach out in the sec-appsec Slack channel.

Samples of PASTA Evaluations

Here is a real sample of an evaluation of the install of a GitLab standalone instance in a hostile environment.

More samples will be added later.

Threat Modeling Within GitLab

The most common use of threat modeling within a tech company like GitLab is our code base. The Security Team has developed a threat modeling "framework" for Engineering with the following in mind:

- Establish a framework that is easy to understand and follow
- The framework should enhance DevSecOps, and introduce minimal overhead
- The framework should be based upon an existing framework with an established track record, if possible, as that may be more familiar to current GitLab team members
- As the Security Team helps with the popular and successful Hackerone bug bounty program, handles compliance needs, and deals with security threats against our assets, having a framework that would allow us to provide "feedback" into the process of analyzing risk is extremely important in helping to determine actual outcomes
- The framework should be flexible enough that it can be used outside of Engineering by other GitLab teams

The purpose of the GitLab Threat Modeling Process is to provide a starting point for future AppSec Code Reviews. Threat models are based on the conceptual architecture of the application

and only utilize code to substantiate findings when possible.

One might ask why we are doing this. Our reasoning involves these important points:

- The Security Team already performs risk analysis and assessments constantly, this gives us a common language
- A common language allows for better communications between Engineering groups, particularly in an all-remote company that makes use of asynchronous communications
- When dealing with partners, customers, and contributors, we have a common point of reference we can use for discussion
- Being a secure company with a mature and robust code base is our goal, and this helps with that effort
- We already have a large code base - instead of trying to rewrite and re-analyze everything from scratch, threat modeling gives us a tool to evaluate new changes to help mitigate future risks. As we triage security issues or introduce new security-related features, we have a tool that will allow us to work off some of the organization and technical debt associated with security and risk is easier-to-digest pieces.

The overall goal is not to create a rigid structure that must be strictly followed, but an adaptive tool that proactively helps uncover security risks before they occur and even chart out solutions based upon the likelihood of the risk.

After evaluation of several popular frameworks, we elected to use the PASTA framework as a base, and with a few minor tweaks for GitLab's environment, we've completed the framework. An overview from the author of the original book can be found here.

We still welcome changes to the framework and deem it to be as much of a living document as the rest of the handbook.

The Framework

Our base framework is PASTA. PASTA is an acronym that stands for Process for Attack Simulation and Threat Analysis. It is a 7-step risk-based threat modeling framework.

There are several other threat modeling frameworks, however others were deemed either too focused on coding or too focused on attacks. PASTA has a much broader range and can easily scale up or scale down as needed, and most other threat modeling frameworks can map into it.

Other threat modeling frameworks examined:

- STRIDE). This has been used by Microsoft, and is primarily focused on threats themselves, and tends to lean toward known/existing threats. As they outgrew STRIDE, they developed SDL (that runs on Microsoft Windows) that allows them to define templates and evaluate threats. We do not run Windows, nor does our focus involve the existing templates they have designed for it. As a part of the overall development process within Microsoft, it is still more "code-centric" that we need.
- Evil Personas. The focus of Evil Personas similar to regular Personas, but the emphasis is on threat actors. it does not cover code-centric projects, just perceived threats. Useful, but limited as it assumes a threat is a person or group of people. Most "persona" scenarios are usually built in or added onto other threat models, refer to this paper on Attack Personas for

a reference to the more aggressive side of personas in threat modeling.
- Playing cards. There are several versions of this including Elevation of Privilege Extremely useful tool, but better designed for in-person collaborations, and is more aligned with STRIDE in mind. Similar to Attack Trees, it focuses more on the attack end in reference to a chunk of infrastructure or code. This would be a fun thing to do at a future Contribute, but it does not scale well for a Zoom-based culture.
- Attack Trees. The focus is on attacks only, as a process to map flaws in existing code and systems.

PASTA has a number of advantages for GitLab over other frameworks:

- Risk centric - mitigating what matters
- Evidence-based threat modeling
  - Harvest threat intel to support threat motives
  - Leverage threat data to support prior threat patterns
- Focus on probability of attack, likelihood, inherent risk, impact of compromise
- Collaborative
- Prioritization should define when and what apps to apply the threat model, and be apart of the threat model process itself

PASTA has the advantage in that it can be adopted from code-based scenarios to infrastructure scenarios easily. It can be adapted to cover non-traditional threats, such as bad PR due to an executive's social media posting or the company's selling of the GitLab product to a controversial organization. It can even be used to map in incident response scenarios, as it allows for threat reinforcement from threat intel sources including logs, intel services, and even previous incidents.

PASTA Stages

There are seven stages to PASTA, here are our definitions. Note that while many of the examples are code-centric, these can be evolved over time to cover non-coding scenarios, such as third-party SaaS application approval, sales strategies and plan into new territories, and so on.

Stage I - Define objectives

This helps set the tone for the project or assessment. It considers the following:

- Business objectives. State the business objective of the project. This can be the purpose of the code, the change to the infrastructure, the goal of the marketing campaign, whatever the objective is.
- Security, Compliance, and Legal requirements. These are both guidelines and limits in a broad sense. For example, the licensing for a proposed third party application should be in order, the privacy policy of the new SaaS application being considered meets muster, or the new code does not require lowering security standards.
- Business impact analysis. This should include impact to mission and business processes, recovery processes, budget impact, and system resource requirements.
- Operational impact. This should include impact to existing processes already in use by operational personnel. For example, extra logging could be introduced that might impact interpretation of system events, increase the number of steps for future changes, or alter documented procedures for advanced troubleshooting.

Stage II - Define technical scope

This does not have to be "tech" per se, but should include project boundaries.

- Boundaries of technical environment. These are the boundaries related to the project itself. This should include the type of data (by classification) that will be touched with this project.
- Infrastructure, application, software dependencies. All of these need to be documented, including the level of impact. For example, if the project requires an upgrade to a subsystem's library and the subsystem has access read-only access to RED data, this needs to be documented as it could impact other users of the subsystem as well as introduce a potential attack vector to RED data.

Stage III - Application Decomposition

This allows mapping of what is important to what is in scope.

- Identify use cases, define application entry points and trust levels even if they are to remain unchanged as a result of the project.
- Identify actors, assets, services, roles, and data sources.
- Document data movement (in motion, at rest) via Data Flow Diagramming. Show trust boundaries, including existing as well as proposed changes.
- Document all data touched and its classification.

Stage IV - Threat Analysis

This is where we mainly look to document relevant threats and threat patterns to the data we have gathered up until now.

- Probabilistic attack scenario analysis, where any scenario that could happen is at least listed.
- Regression analysis on security events, where we example events that touch on some of the same or similar components.
- Threat intel correlation, by taking data from logs, Hackerone reports, incidents, and other sources that can be used to indicate an attack scenario.

Stage V - Vulnerability and weakness analysis

This is were we examine the threats, ranking them and assessing their likelihood.

- Examine existing vulnerability reports and issues tracking.
- Threat to existing vulnerability mapping using threat trees.
- Design Flaw analysis using use and abuse cases.
- Scorings (CVSS) and Enumeration (CWE/CVE).
- Impacted systems, sub-systems, data. Are we adding to or altering something that has a history of exploitation. Is the current state vulnerable, and will this change potentially be influenced by or change that assessment.

Stage VI - Attack modeling

We go through the threats and turn them into attacks, by examining the attack surface before and after our proposed changes.

- Attack surface analysis for the impacted components.
- Attack tree development. This is where something like MITRE ATT&CK can assist.
- Attack --> Vulnerability --> Exploit analysis using attack trees.
- Summarize the impact and explain each risk.

Stage VII - Risk and Impact Analysis

This covers the development of the rationale for mitigation based upon residual risk.

- Qualify and quantify the business impact.
- Countermeasure identification and residual impact.
- Identify risk mitigation strategies. Effectiveness of mitigation(s) vs cost to implement mitigation.
- Residual benefits, as the implementation of a change to single component as a mitigation effort could mean increased security for other systems that access that component.

Three Tiered Approach

To help with implementing and using the PASTA framework, we can use a three-tiered approach. This approach is as follows:

Blind threat model

GitLab's best practices applied to components of the project.

- Maps key goals of app or service and correlates to clear technical standards for architecture, hardening of server/service, app framework, containers, etc.
- Best practices per component. For example, TLS settings that are set to a GitLab standard, noting if our own standard is higher or lower than industry best practices.
- Best practices for coding are applied here as well, the "Sec" part of DevSecOps and our integration of this into CI/CD.
- SAST/DAST policies and scopes. We can "eat our own dogfood" to improve the quality of the changes we implement.

Applies Stage I & Stage II of PASTA

Evidence Driven threat model

Proof of a threat via numerous indicators as opposed to just theory or conjecture.

- Integrate threat log data analysis.
- Focus on logs that support attack vector with the greatest motives (e.g. TLS MITM vs Injection-based attacks).
- Correlate threat intel for foreseeing trends of attacks for target applications that are related to project components.

Applies Stage III, IV, V of PASTA

Full Risk Based Threat Model

- Statistical/probabilistic analysis on threat data & attack effectiveness
- Consider non-traditional threat vectors
- This includes threat intel, H1 trends, existing logs
- Substantiate the threats that are defined with real data

Applies all stages of PASTA

Additional Resources

Here are some helpful links.

- Excerpt from a Security Department "Show and Tell" discussing Threat Modeling.
- Blog post by Michael Henriksen that talks about using Draw.io available via diagrams.net to construct diagrams and flowcharts, and using them during threat modeling. Included is a link to useful libraries for threat model diagrams.
- In addition to Elevation of Privilege there is also OWASP Cornucopia, which leans more towards web-based applications.
- MITRE ATT&CK. This is not a framework used for threat modeling per se, but it could be adapted to and mapped to an existing threat modeling framework.

SECTION: security/product-security/application-security/threat-modeling/howto.md

TL;DR

In very, very short conclusion:

Threat modeling should and can be done at almost any point in the development process, but really you should start it early in the process. You should take some "what-could-possibly-go-wrong" or attacker mentality and apply it to your feature.

For the impatient let's have the maybe shortest possible threat modeling guide:

- Draw a diagram of your feature and point out:
  - Where does untrusted input come from?
  - Where are zones of different trust levels and where are the trust boundaries between those?
- Take an attackers perspective and assume worst cases to define the threats.
  - Try to order by most likely and impactful threats first.
- Document the threats and map them back to your feature. Create follow-up issues with directly responsible individuals and due dates.

What is Threat Modeling

Let's keep it short and simple here by just taking the initial sentence from our Threat Modeling handbook page:

> Threat modeling is the process of taking established or new procedures, and then assessing it

for potential risks.

This is maybe the most high-level and abstract description of threat modeling, now let's put it to some practical use. We take something, an established or new process, really this might be anything. Like our own assessment-tool, a stand-alone GitLab instance or maybe a new piece of infrastructure (internal link).

For the rest of this HowTo let's call the thing we're doing a threat model for a feature because for GitLab most of the threat modeling which should be done would be on features of the product or features of the infrastructure of our SaaS offering.

What a threat model does, it let's you take apart and decompose the feature you're looking at so that you can, in some more or less formalized way, identify and describe threats towards the feature and its components. This description so far sounds really formal and might not help you to actually come up with a threat model, just take it as a vague baseline of what (identify threats) should be achieved and let's now see how we can get there.

The formal framework that GitLab uses is called PASTA. This guide helps apply the PASTA principals in a lightweight manner to maximize their impact when followed by any GitLab team member.

When and where to start

You might wonder: "When should I start to do some threat modeling on the feature I'm planning to implement?". It's really not obvious but luckily it's never too late and never too early to create a threat model for any given feature which is currently being developed or used. Also, like everything in security, it's more of a process than a fixed, eternal state. A threat model needs to be adopted and refined if the feature changes or even if the feature stays the same but it's being used in a different context or environment. So yes, this is some kind of "extra" work in a sense that a proper threat model just doesn't create itself. But those extra steps will pay off soon enough by yielding more insights about the "what could possibly go wrong" moments which are also known as threats. Even tough it's never too late it will pay off even more if you start the process early and keep the threat model up to date for any additions and changes to the feature. This is also due to the fact that the fixing of insecure design decisions might be rather complex and even disruptive(internal link).

Tools: ·and·

Most threat modeling frameworks rely on some diagrams to be drawn and there's a lot of quite heavy tooling around this. Such tools would take some data flow diagram (DFD) and automatically map certain threats to certain components based on what they are. This can be really useful in some cases, but at GitLab we need some more flexibility as the features we're looking at often would not really fit well into the strict schematics of e.g. STRIDE) based threat modeling and thus the output might not yield much meaningful threats.

Drawing diagrams

For diagrams there are almost endless possibilities, for easy and integrated usage within GitLab it'd be recommended to use one of our supported diagram tools. But really to keep it flexible and accurate you can use whatever diagramming tool you're comfortable with. It should

just deliver an accurate diagram of the feature. For instance this mermaid diagram of our Kubernetes Agent is completely sufficient as a start. It shows all involved components in their interaction at a level which is still understandable and meaningful.

```
mermaid
graph TB
  agentk -- gRPC bidirectional streaming --> nginx

  subgraph "GitLab"
  nginx -- proxy pass port 5005 --> kas
  kas[kas]
  GitLabRoR[GitLab RoR] -- gRPC --> kas
  kas -- gRPC --> Gitaly[Gitaly]
  kas -- REST API --> GitLabRoR
  end

  subgraph "Kubernetes cluster"
  agentk[agentk]
  end
```

If we'd keep it too simple like:

```
mermaid
graph TB
  agentk -- talks to --> GitLab

  subgraph "Kubernetes cluster"
  agentk[agentk]
  end
```

It'd still not be wrong but we wouldn't be able to deduce too many detailed threats from this.

Trust and boundaries

One notable component which typically does not come with your typical architecture diagram is a trust boundary. A trust boundary is also not depicted explicitly in the Kubernetes Agent diagram, but we can deduce it. A trust boundary is separating parts in the diagram which have different levels of trust. In the Kubernetes Agent case we can see this clearly, the Kubernetes cluster can be anywhere and be controlled by anyone, it's not really trustworthy. The GitLab controlled components however are controlled by GitLab, therefore very much trusted. So in conclusion we have a trust boundary between those two parts of the diagram. This now is the part where the actual threats come into play. The threats typically manifest at those trust boundaries. A first threat which might come to mind when just looking at this trust boundary:

- The communication between agentk and kas might be unencrypted

This is a very simple and generic thing to consider, still a legit concern and a first step for

a threat model.

Finding the threats

For covering more of the threat ground we need to shift a bit the perspective towards a "worst-case" or "what could possibly go wrong" attitude, this generally helps a lot to find more threat scenarios. In the very formalized STRIDE approach the actual threat classes are:

- Spoofing
  - Impersonating something or someone else
- Tampering
  - Modifying data or code
- Repudiation
  - Claiming to have not performed an action
- Information disclosure
  - Exposing information to someone not authorized to see it
- Denial of Service
  - Deny or degrade service to users
- Elevation of Privilege
  - Gain capabilities without proper authorization

Hence the name STRIDE. While we do not use the formal STRIDE framework, we can use these threat classes to get an idea on what should be considered to define the concrete threats in our model.

So spoofing in the Kubernetes Agent example might manifest in one agentk being able to impersonate another. The example from above "The communication between agentk and kas might be unencrypted" would for instance fit in the information disclosure threat class. It's not a matter of thinking about every possible single thing which might go wrong here. One could easily come up with an overwhelming amount of potential and rather obscure threats, but that might be very distracting and we would miss the point of the threat model. We primarily want to aim for those threats which seem likely to occur and those which are really impactful when they occur.

Consider both .com and self-managed

Make sure to think a bit out of the box: a certain feature might work well on GitLab.com but cause trouble and outages in some self-managed environment which is set up quite differently than our SaaS platform. This can be a threat as well and it's caused due to some environmental change for our feature. This in conclusion means that we should always consider the environment our feature is being used in. That environment might change over time or in different deployment scenarios, and it might not always be friendly and trusted in the first place.

What's next?

Once the initial steps have been done and we have a first set of well thought out threats we can now use this list during development of our feature to mitigate what we've deemed to be the threats and worst case scenarios.

Documenting the Threat Model

The Stable Counterpart should document the threat model and the results. The threat model will be added to the AppSec Threat Models repository(internal link). This also includes templates for threat modeling that anyone can use in issues or epics.

Ensure Ownership of Threats

Consider creating an issue with a living description which summarises the threat model each time its updated, and links off to issues for each threat, like so:

Threat	Comments
Unencrypted communication between agentk and kas	· grpc communication is done over TLS encrypted Websockets (see 123)
agentk might be able to impersonate another cluster's agentk	Issue 124 to review authorization
Attacks on gitaly level	agentk has indirect access to gitaly via kas, this might be abused for injections or IDOR attacks	Issue 125 - check data flows from agentk towards gitaly

Each threat should have an Issue created where a proposal to avoid, prevent, detect, or recover from the threat is discussed by the team. These issues should have an Assignee and a Milestone or Due Date. Initially the Assignee should be the Project Manager, who will prioritise and re-assign the issue as appropriate. The Stable Counterpart's role is to help create that proposal, to help the team understand and address the issue, and to review how the threat is mitigated pre-merge.

Also remember to consider making these issues public if they are not describing a currently implemented risk; issues that discuss how to improve upon the security posture of an implementation can be made public by default.

Iterating

A threat model is never really done. The team should be familiar with the threat model and request another App Sec Review or an update to the threat model when new features or changes arise. The Stable Counterpart should work closely with the team to